LAB 9

Design Analysis Report

Health Insurance Claim Portal

Name	Gagan Sharma
Roll No	202401100200140
Branch / Section	CSE B
Experiment	Lab 9 - Design Analysis Report

1. Introduction

The Health Insurance Claim Portal is a web-based system that lets policyholders submit, track and manage medical insurance claims online, while allowing hospitals, claim officers and administrators to verify documents, assess claims and settle payments. The goal of this report is to analyse the design of such a portal: its requirements, architecture, major components, data model, design principles, and quality attributes.

1.1 Objectives

- Replace paper-based claim filing with a transparent, trackable digital workflow.
- Reduce claim processing time through automated validation and rule-based checks.
- Protect sensitive personal and medical data (privacy, integrity, auditability).
- Provide a modular design that is easy to maintain, test and extend.

2. Requirement Analysis

2.1 Stakeholders and Actors

Actor	Role in the system
Policyholder	Registers, views policy, submits claims, uploads documents, tracks status, raises disputes.
Hospital / TPA	Submits pre-authorisation and cashless claim requests, uploads bills and discharge summaries.
Claim Officer	Reviews documents, verifies coverage, approves / rejects / queries claims.
Administrator	Manages users, policies, plans, rules and system configuration; views reports.
Payment Gateway	External system that transfers approved claim amounts.

2.2 Functional Requirements

- User registration, login, profile and role-based access control.
- Policy lookup: coverage, sum insured, waiting periods, exclusions.
- Claim submission (reimbursement and cashless) with multiple document upload.
- Automated eligibility and duplicate-claim checks.
- Claim workflow: Submitted, Under Review, Query Raised, Approved, Rejected, Paid.
- Notifications by email / SMS at every status change.
- Payment settlement and downloadable claim statements.
- Admin dashboard and reports (claim ratio, turnaround time, fraud flags).

2.3 Non-Functional Requirements

Attribute	Requirement
Security	Encryption in transit (TLS) and at rest, MFA for staff, audit logs.
Performance	Page response under 2 s; support 5,000+ concurrent users.
Availability	99.5% uptime; daily backups and disaster recovery.
Scalability	Horizontal scaling of stateless services.

Attribute	Requirement
Usability	Responsive, accessible UI with simple multi-step claim form.
Compliance	Data-protection and insurance regulatory guidelines (e.g., IRDAI, DPDP Act).

3. System Architecture

The portal follows a **layered, service-oriented architecture** so that presentation, business logic and data access are separated. This improves maintainability and allows each layer to be scaled or replaced independently.

Layer	Components	Responsibility
Presentation	Web app (React/HTML), mobile-responsive UI	Forms, dashboards, status tracking, input validation.
API Gateway	REST API, authentication filter, rate limiter	Single entry point; routing, JWT validation, throttling.
Business / Service	Auth, Policy, Claim, Document, Notification, Payment, Fraud-check services	Core workflow and business rules.
Data Access	Repositories / ORM	CRUD operations, transactions, query abstraction.
Data Storage	Relational DB, object storage, cache	Structured data, scanned documents, session / lookup caching.
External	Payment gateway, SMS / email provider, hospital APIs	Third-party integrations through adapters.

Request flow: Browser, then API Gateway (authentication), then Claim Service (validation and rules), then Repository / Database, with the Notification Service informing the user asynchronously through a message queue.

4. Design Analysis

4.1 Major Classes and Responsibilities

Class	Key attributes	Key methods
User	userId, name, email, role, passwordHash	login(), logout(), updateProfile()
Policy	policyNo, planType, sumInsured, startDate, endDate	isActive(), getCoverage(), getBalance()
Claim	claimId, policyNo, amount, type, status, dateFiled	submit(), changeStatus(), calculatePayable()
Document	docId, claimId, type, filePath, uploadedAt	upload(), verify(), download()
ClaimOfficer	officerId, department	review(), approve(), reject(), raiseQuery()
Payment	paymentId, claimId, amount, mode, status	initiate(), confirm(), refund()
Notification	notifId, userId, message, channel	send(), markRead()

4.2 Relationships

- A **User** (policyholder) owns one or more **Policies** (1 to many).
- A **Policy** can have many **Claims**; each **Claim** belongs to exactly one Policy.
- A **Claim** has many **Documents** (composition: documents cannot exist without the claim).
- A **Claim** is reviewed by one **ClaimOfficer** and results in at most one **Payment**.
- **ClaimOfficer** and **Policyholder** inherit from **User** (generalisation).

4.3 Claim Lifecycle (State Design)

Current state	Event	Next state
Draft	Policyholder submits	Submitted
Submitted	Auto-validation passed	Under Review
Under Review	Documents missing	Query Raised
Query Raised	User responds	Under Review
Under Review	Officer approves	Approved
Under Review	Officer rejects	Rejected
Approved	Payment successful	Paid / Closed

5. Database Design

A relational database (e.g., MySQL / PostgreSQL) is chosen because claim data is structured and needs ACID transactions. The schema is normalised up to 3NF to avoid redundancy.

Table	Primary key	Important columns / foreign keys
users	user_id	name, email (unique), password_hash, role
policies	policy_no	user_id (FK), plan_type, sum_insured, start_date, end_date
claims	claim_id	policy_no (FK), officer_id (FK), amount, status, filed_on
documents	doc_id	claim_id (FK), doc_type, file_path, verified
payments	payment_id	claim_id (FK), amount, mode, txn_ref, status
audit_log	log_id	user_id, action, entity, timestamp

Indexes are placed on *claims.status*, *claims.policy_no* and *users.email* to speed up the most frequent lookups.

6. Design Principles and Patterns Applied

6.1 SOLID Principles

Principle	Application in the portal
Single Responsibility	Each service (Claim, Payment, Notification) has one reason to change.
Open / Closed	New claim types or fraud rules are added as new classes without modifying existing code.
Liskov Substitution	Any PaymentMethod subclass (UPI, NetBanking, Card) can replace the base type.

Principle	Application in the portal
Interface Segregation	Small interfaces such as Notifier and PaymentProcessor instead of one large interface.
Dependency Inversion	Services depend on abstractions (repository interfaces), not concrete databases.

6.2 Design Patterns

Pattern	Where used	Benefit
MVC	UI, controller and model separation	Clean separation of concerns.
Factory	Creating different claim objects (cashless / reimbursement)	Centralised object creation.
Strategy	Claim amount calculation and payment modes	Swap algorithms at runtime.
State	Claim status transitions	Prevents invalid transitions.
Observer	Notifications on status change	Loose coupling between claim and notifier.
Repository	Data access layer	Database independence, easier testing.
Adapter / Facade	Payment gateway and hospital API integration	Hides third-party complexity.

7. Security Analysis

- **Authentication:** hashed passwords (bcrypt), JWT sessions, multi-factor authentication for officers and admins.
- **Authorisation:** role-based access control; a policyholder can only view their own claims.
- **Data protection:** TLS for transport, AES-256 encryption for stored documents and sensitive fields.
- **Attack prevention:** parameterised queries (SQL injection), output encoding (XSS), CSRF tokens, file-type and size checks on uploads.
- **Auditability:** every view, edit and approval is recorded in the audit log.
- **Fraud control:** duplicate-claim detection, abnormal-amount flags and hospital blacklist checks.

8. Quality Attribute Analysis

Quality	Design decision supporting it	Possible trade-off
Maintainability	Layered architecture, modular services	More initial design effort.
Scalability	Stateless services, load balancer, caching	Cache consistency handling.
Performance	Indexes, async notifications via queue	Eventual consistency for notifications.
Reliability	Transactions, retries, backups	Added infrastructure cost.
Security	Encryption, RBAC, audit logs	Slight performance overhead.
Testability	Interfaces and dependency injection	More abstractions to manage.

8.1 Risks and Mitigation

Risk	Mitigation
Data breach of medical records	Encryption, access control, regular security audits.
Payment gateway downtime	Retry queue and fallback payment mode.
Large document uploads slowing the system	Object storage with direct upload and size limits.
Fraudulent claims	Rule-based and analytics-driven fraud detection.

9. Testing Strategy

- **Unit testing:** business rules such as eligibility check and payable amount calculation.
- **Integration testing:** claim service with database, payment gateway sandbox and notification service.
- **System / UAT:** end-to-end claim from submission to payment for each actor.
- **Security and load testing:** penetration tests and concurrent-user simulations.

10. Conclusion

The analysis shows that a layered, service-oriented design built on SOLID principles and well-known design patterns suits a Health Insurance Claim Portal. Separating the presentation, business and data layers gives maintainability; the State and Observer patterns model the claim lifecycle and notifications cleanly; and strong security controls protect sensitive medical data. Together, these design choices give a portal that is secure, scalable, easy to extend (for example, adding AI-based fraud detection or new insurers) and convenient for all users.

References

- Sommerville, I., *Software Engineering*, Pearson.
- Gamma, Helm, Johnson, Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*.
- Pressman, R., *Software Engineering: A Practitioner's Approach*, McGraw-Hill.